COMPSCI 130 S1 2022
Introduction to Software Fundamentals

Binary Search Trees – Deletion and BST Performance

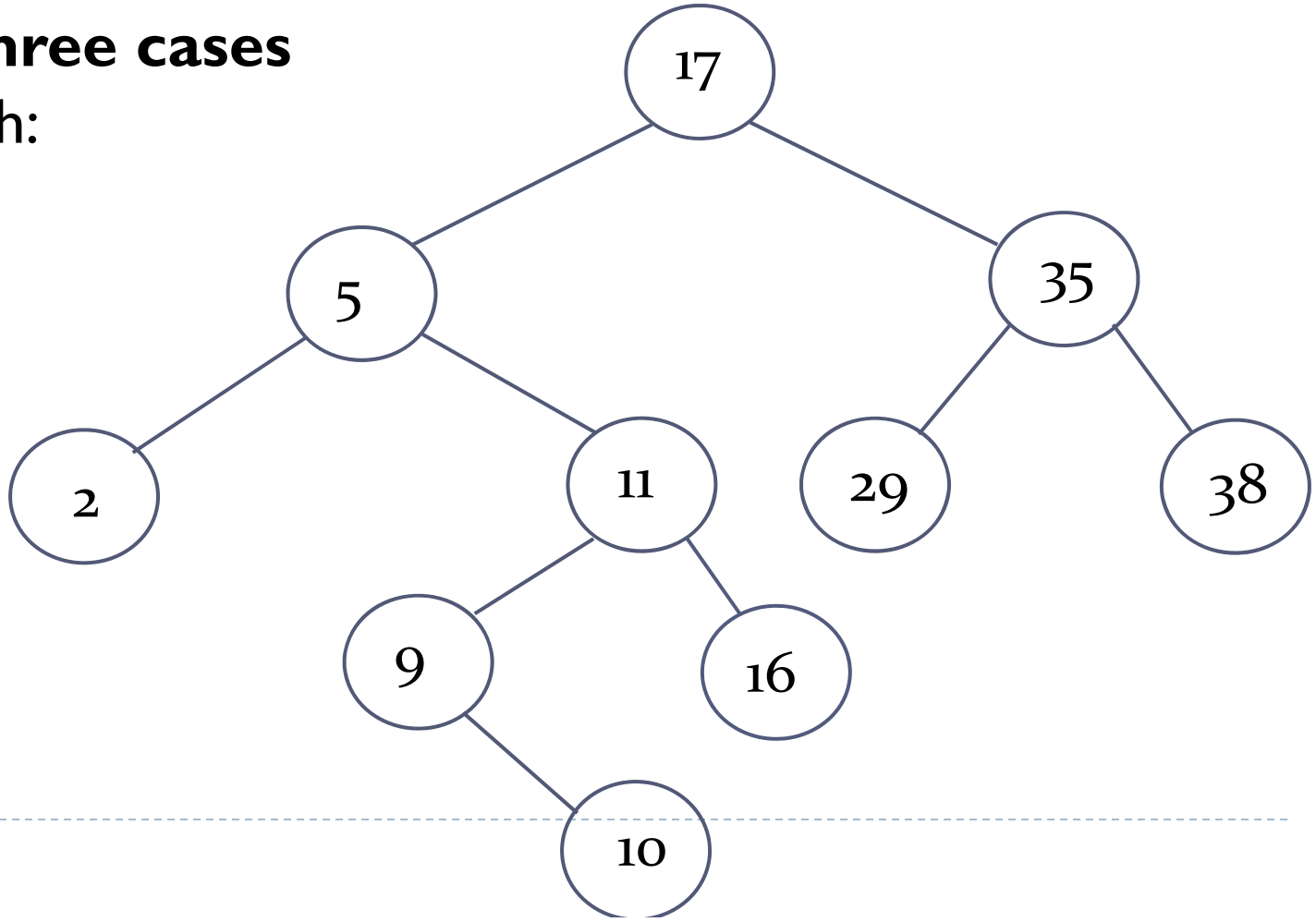
Agenda & Reading

- Agenda
  - Deleting values from binary search trees
    - What is the in-order successor
  - Binary Search Tree Performance
    - Advantages of Binary Search Trees

- Online Coursebook:
  - https://onlinetextbook.auckland.ac.nz/11-binary-search-trees/

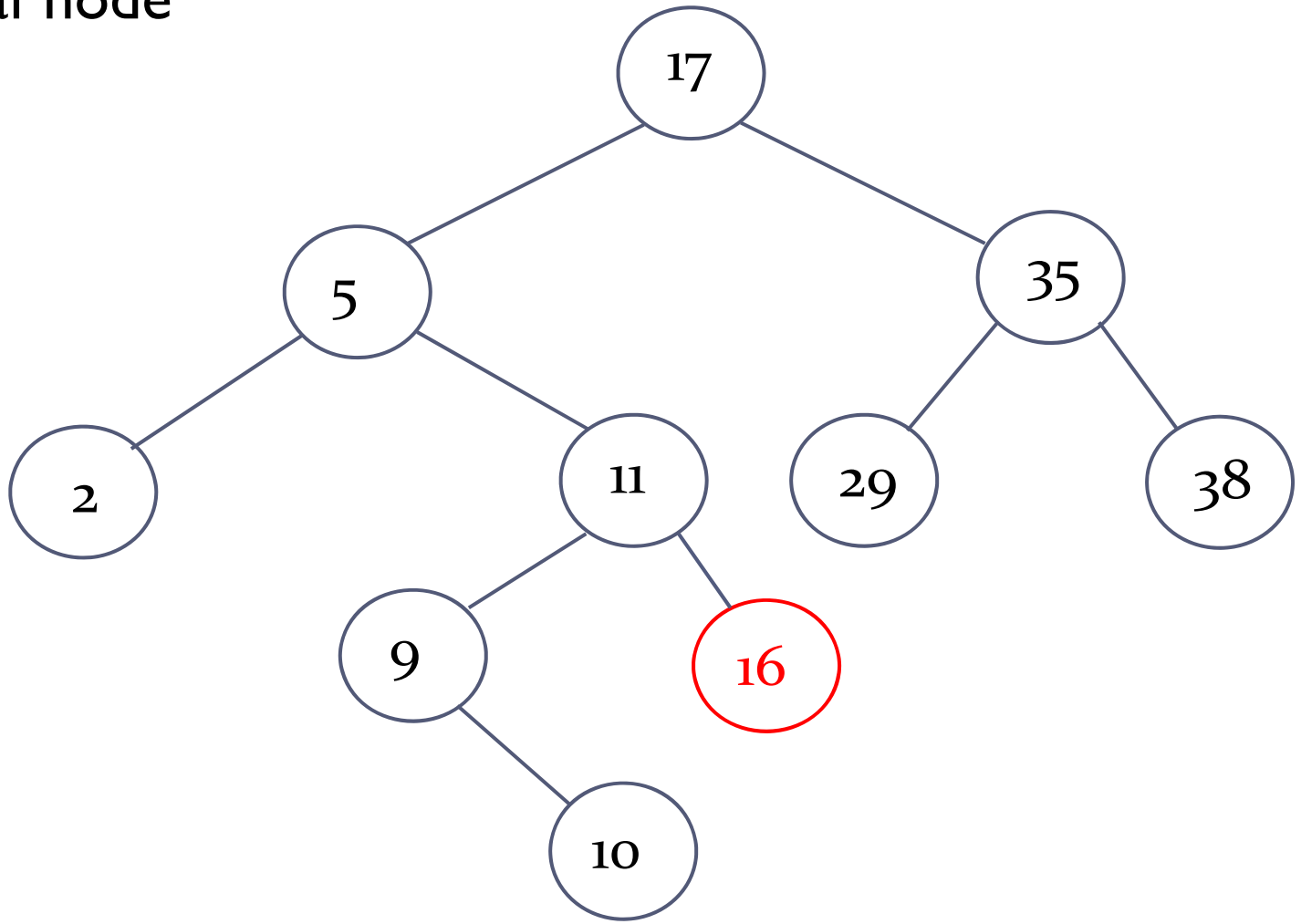
Deleting values from Binary Search Trees

- Deleting nodes is a little bit trickier than insertion
  - We must maintain the binary search tree property!
- There are **three cases** to distinguish:



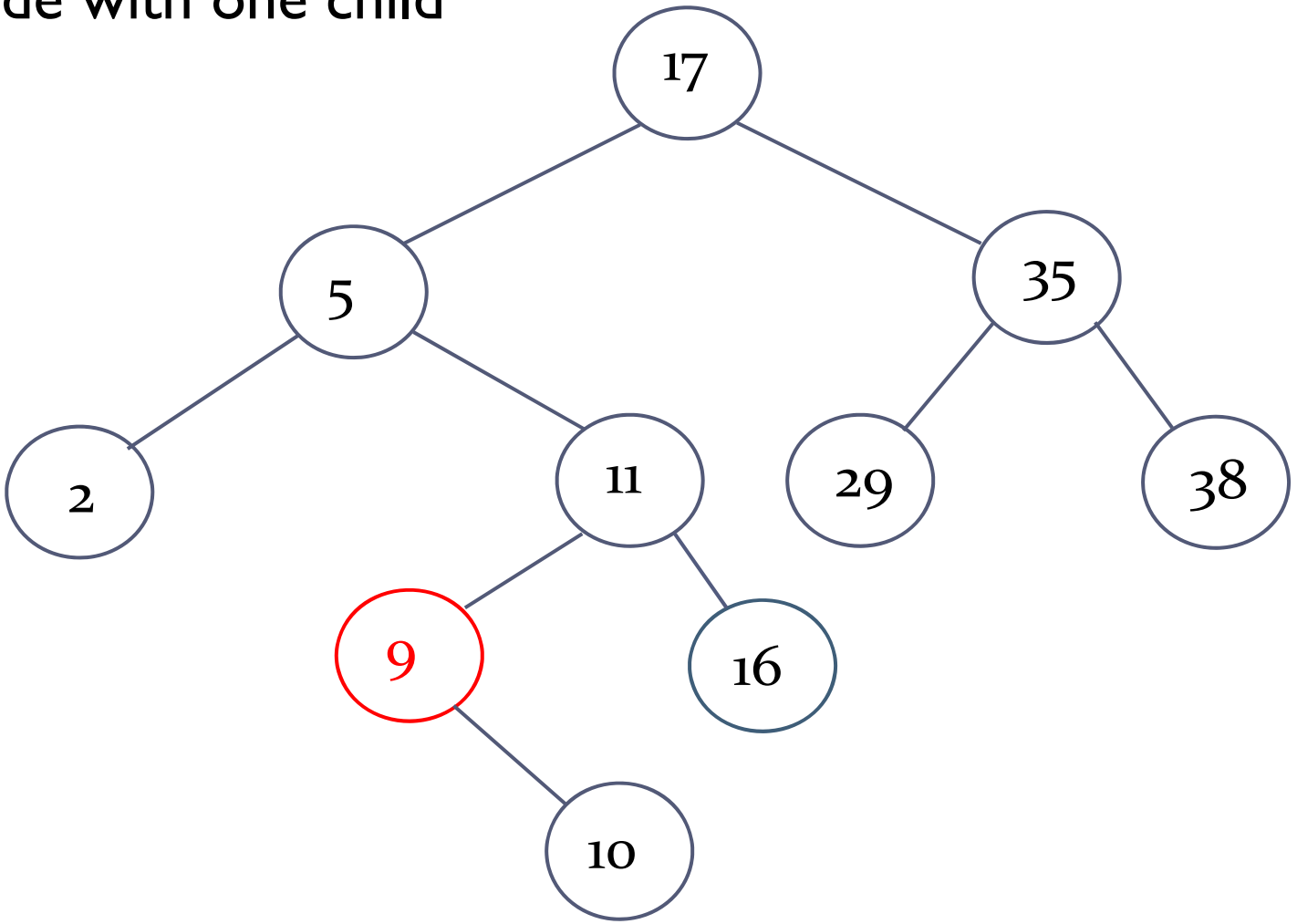
Binary search trees - deleting

1. Deleting a leaf node



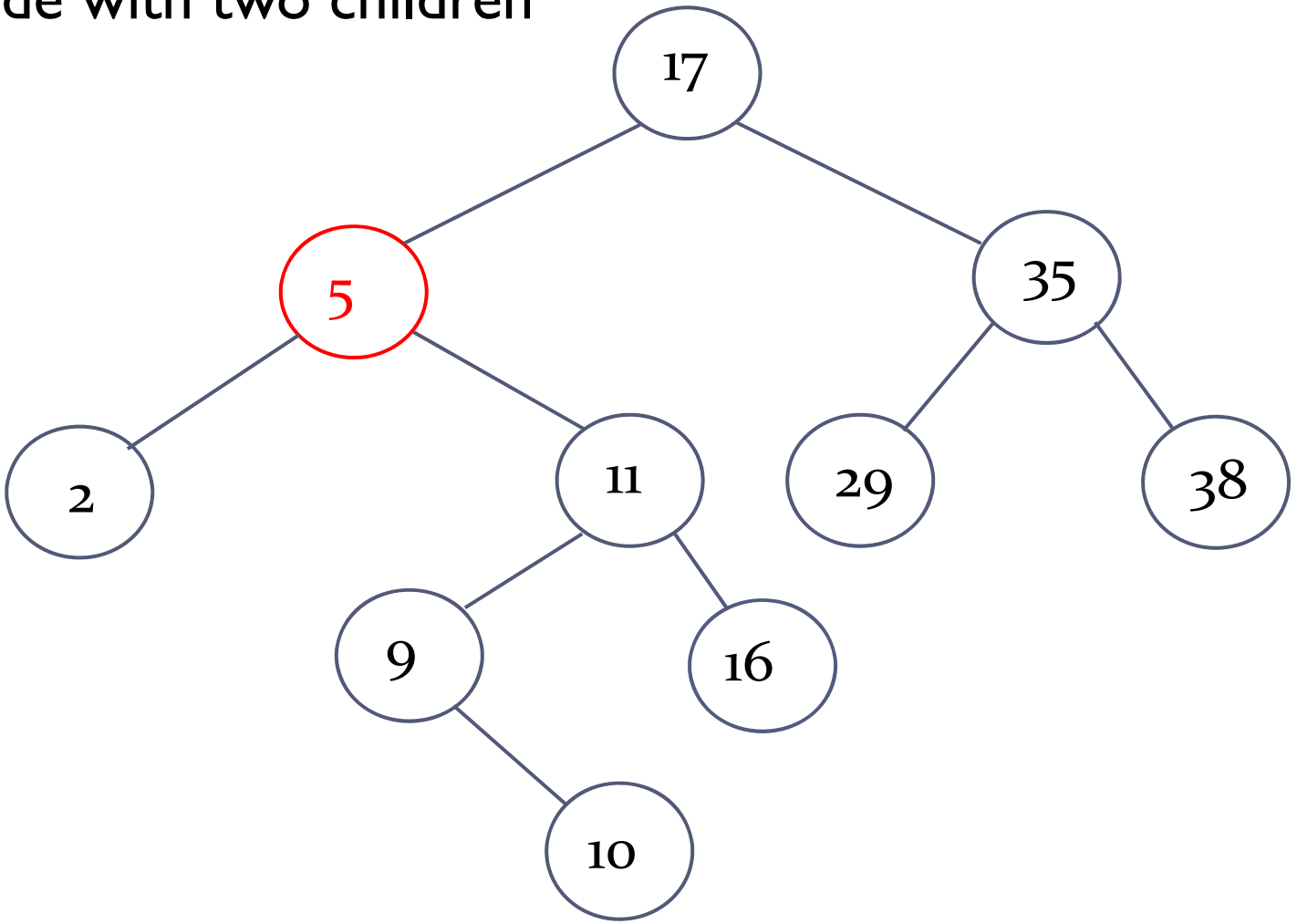
Binary search trees - deleting

2. Deleting a node with one child



Binary search trees - deleting

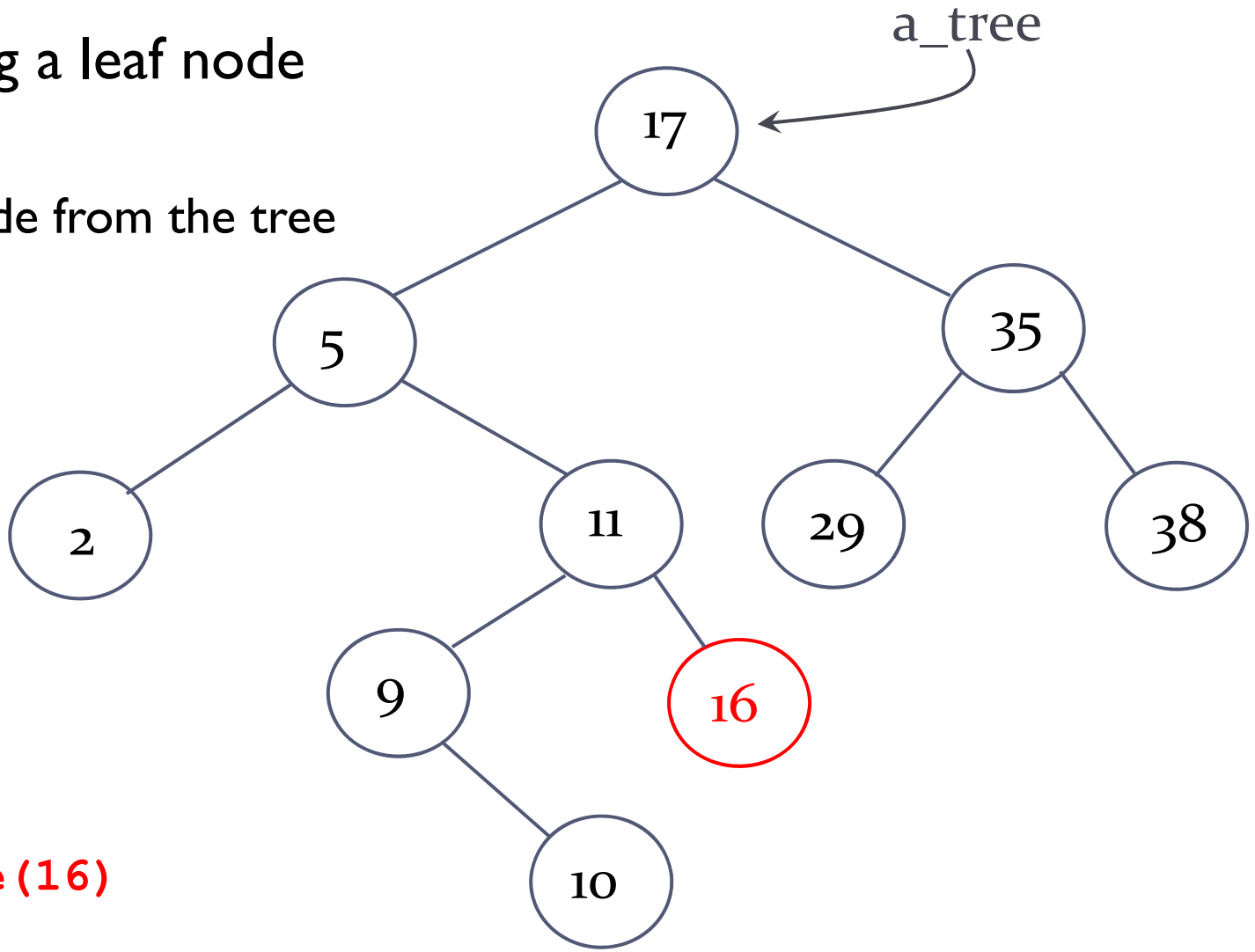
3. Deleting a node with two children



Binary search trees - deleting

Case 1: Deleting a leaf node

- Find the node
- Remove the node from the tree

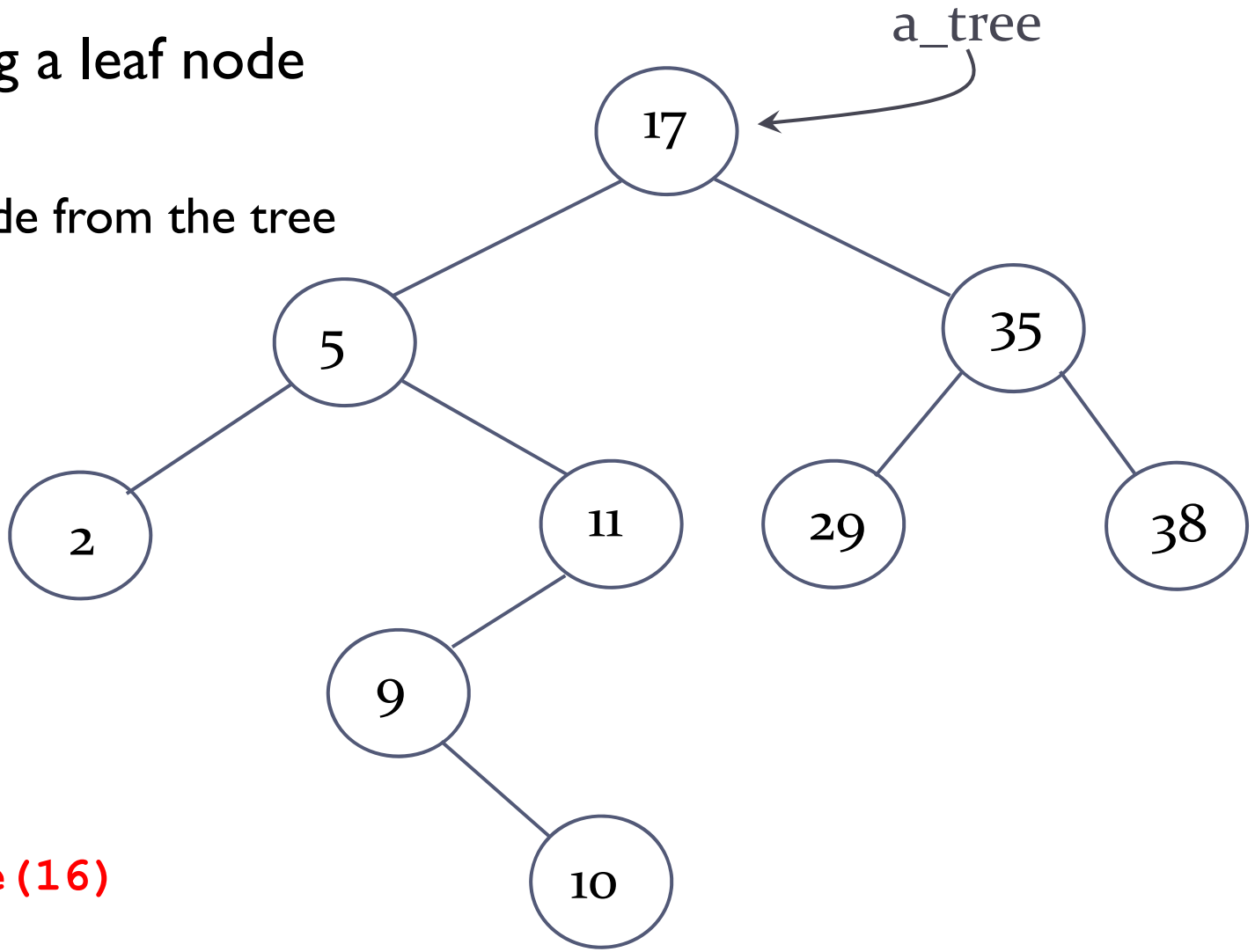



```
a_tree.delete(16)
```

Binary search trees - deleting

Case 1: Deleting a leaf node

- Find the node
- Remove the node from the tree

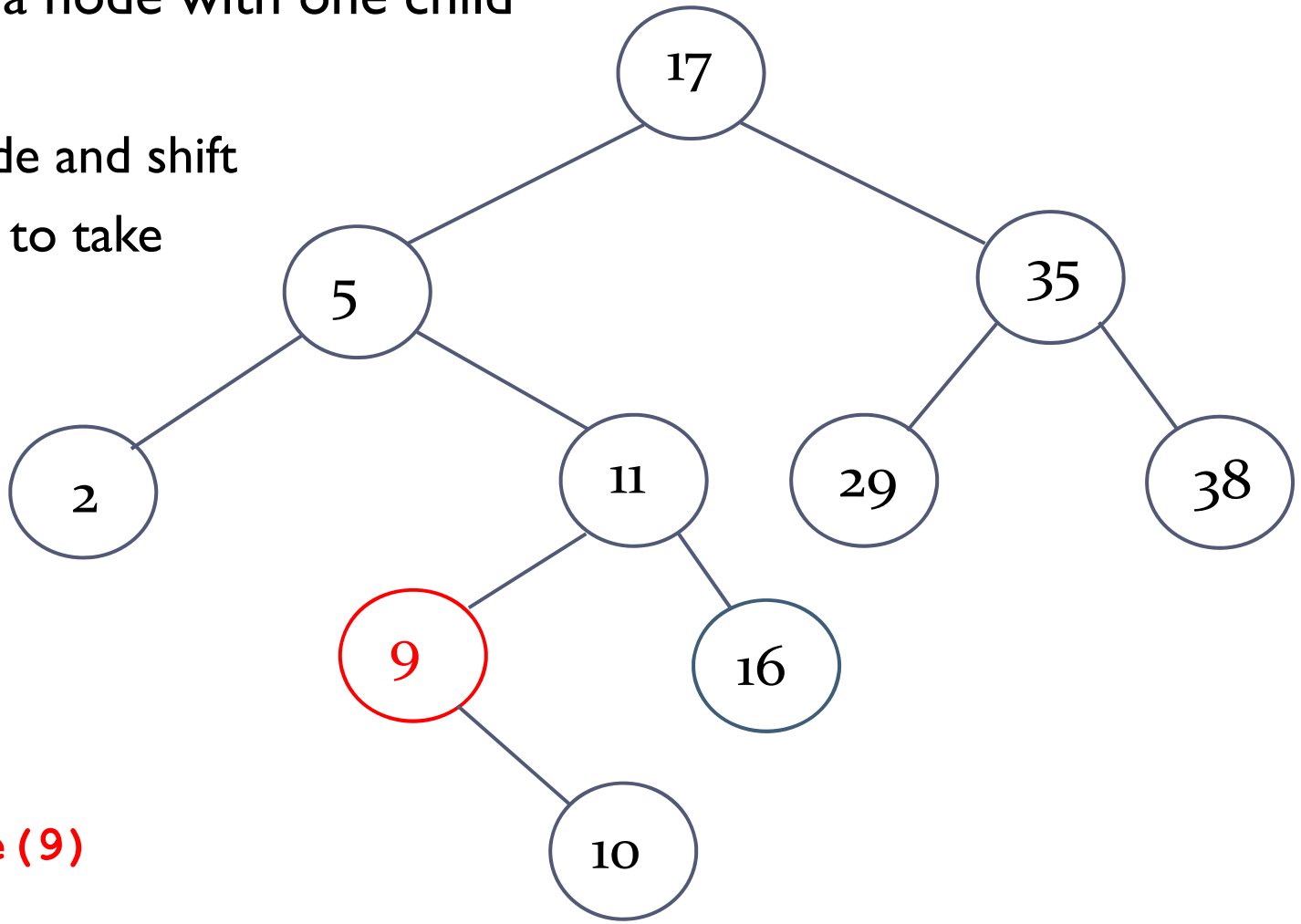



```
a_tree.delete(16)
```

Binary search trees - deleting

Case 2: Deleting a node with one child

- Find the node
- Remove the node and shift its only child up to take its place

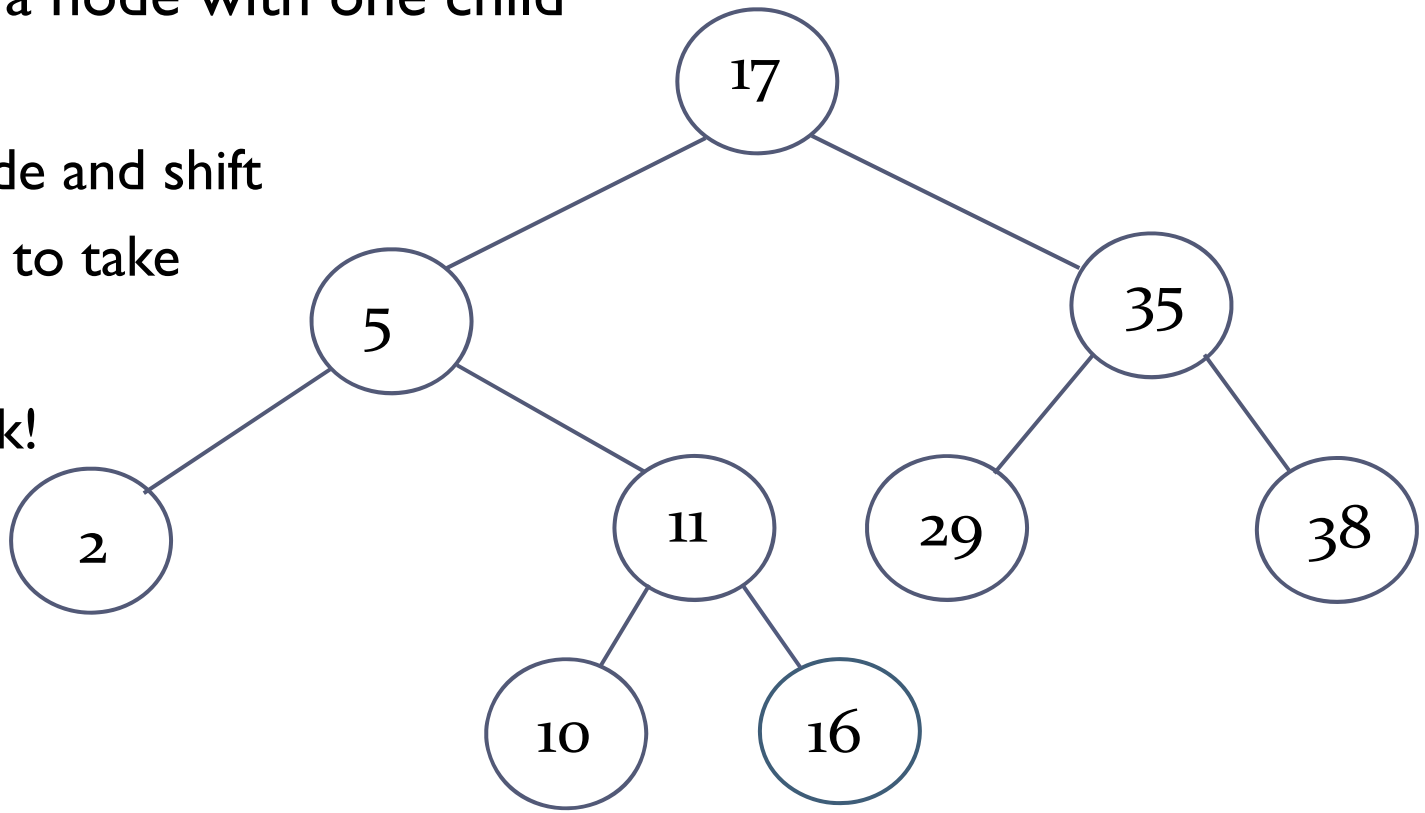



```
a_tree.delete(9)
```

Binary search trees - deleting

Case 2: Deleting a node with one child

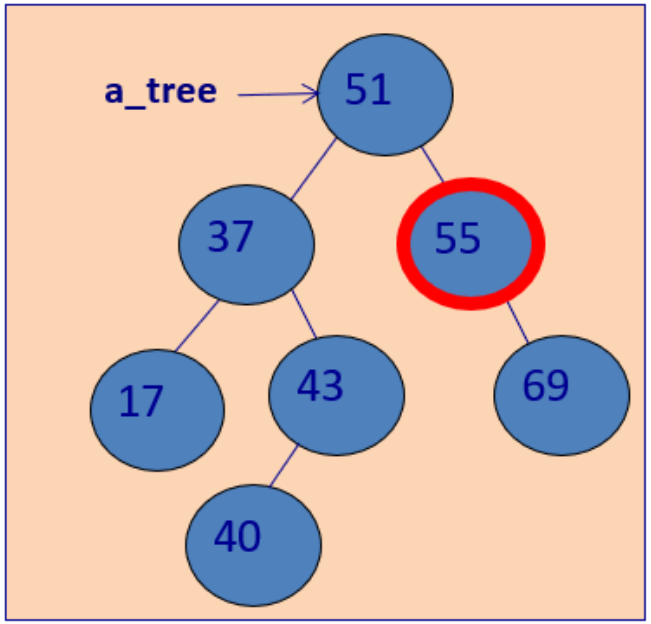
- Find the node
- Remove the node and shift its only child up to take its place
- BST property ok!




```
a_tree.delete(9)
```

Binary search trees - deleting

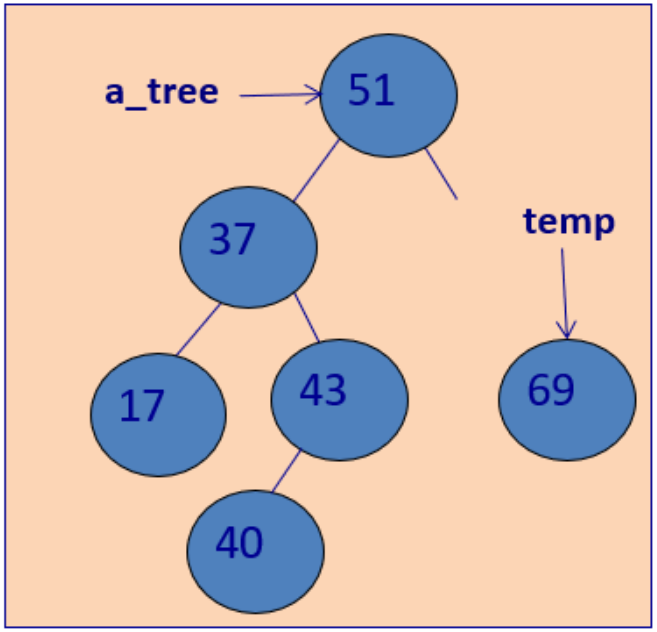
- **Case 2:** Deleting a node with one child
- Another example




```
a_tree.delete(55)
```

Binary search trees - deleting

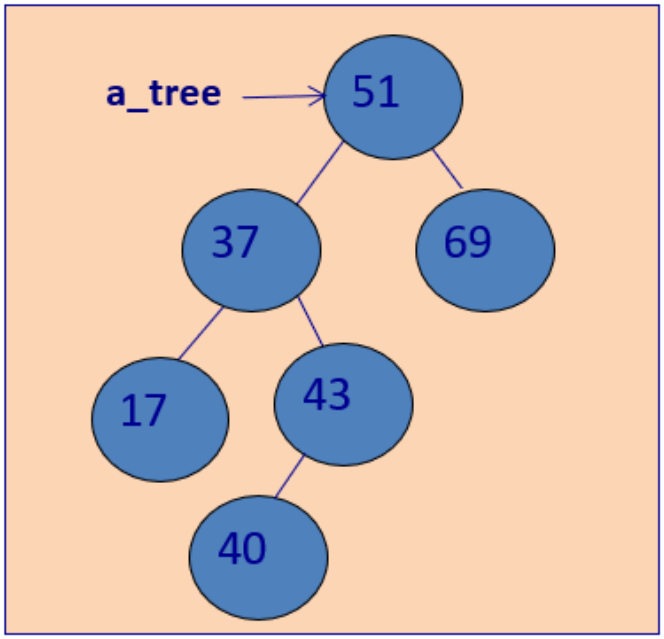
- **Case 2:** Deleting a node with one child
- Another example




```
a_tree.delete(55)
```

Binary search trees - deleting

- **Case 2:** Deleting a node with one child
- Another example

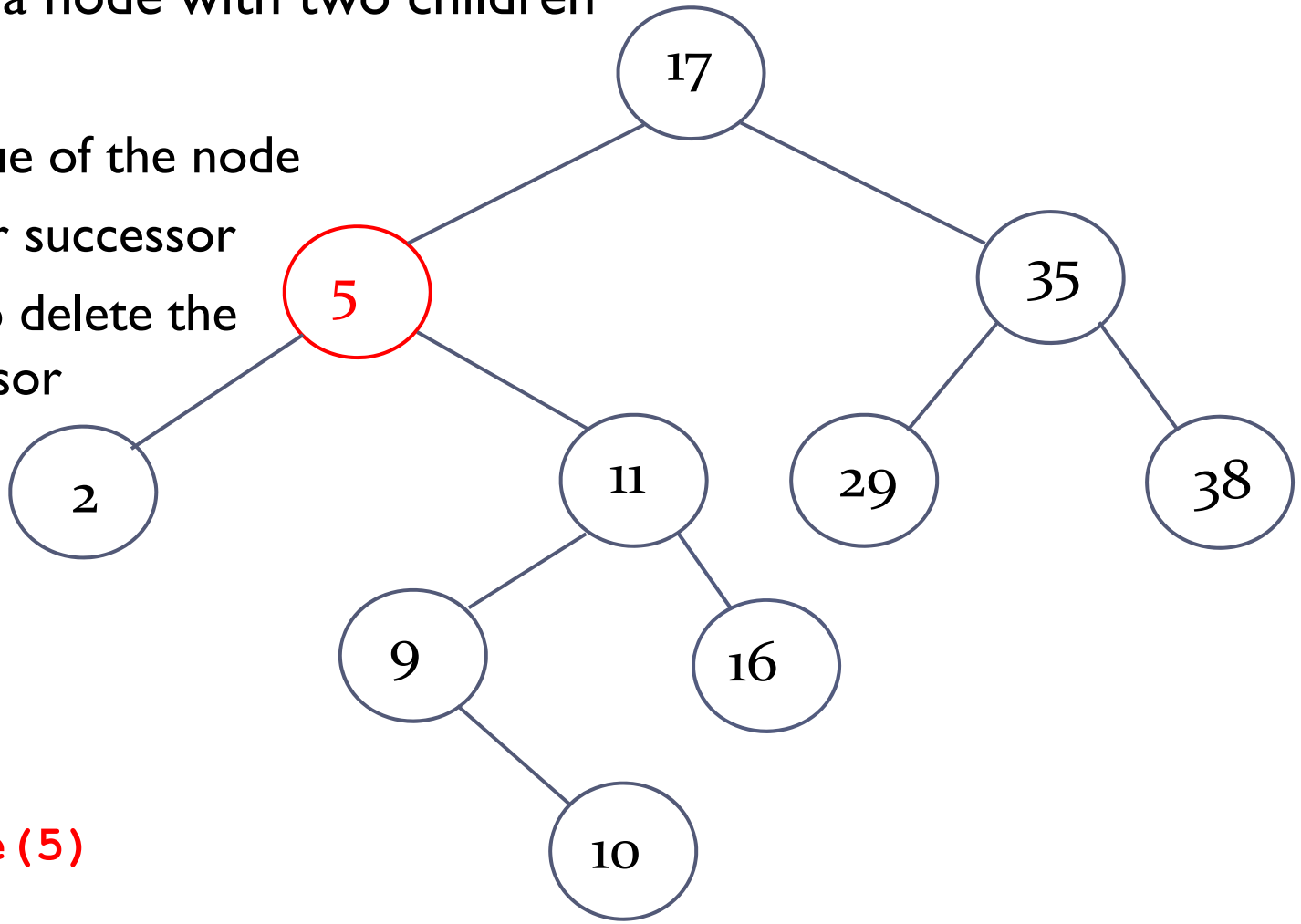



```
a_tree.delete(55)
```

Binary search trees - deleting

Case 3: Deleting a node with two children

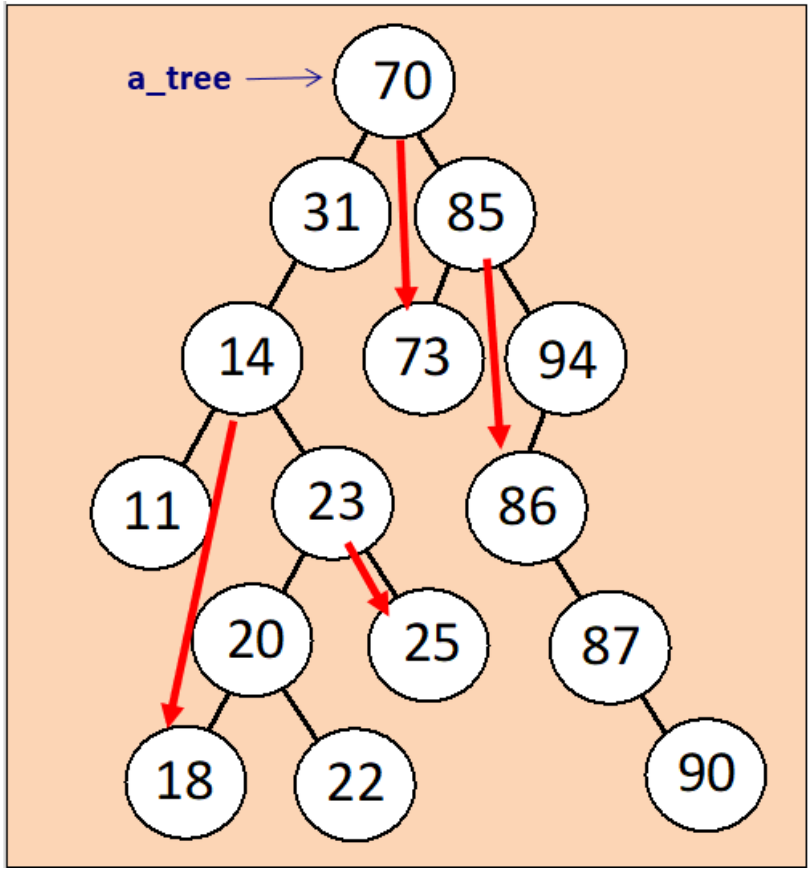
- Find the node
- Replace the value of the node with its in-order successor
- We also have to delete the in-order successor




```
a_tree.delete(5)
```

What is the in-order successor?

The next biggest value when an in-order traversal is performed on the subtree with the node as its root.



How do we find the in-order successor of a node?

The in-order successor of 85?

The in-order successor of 23?

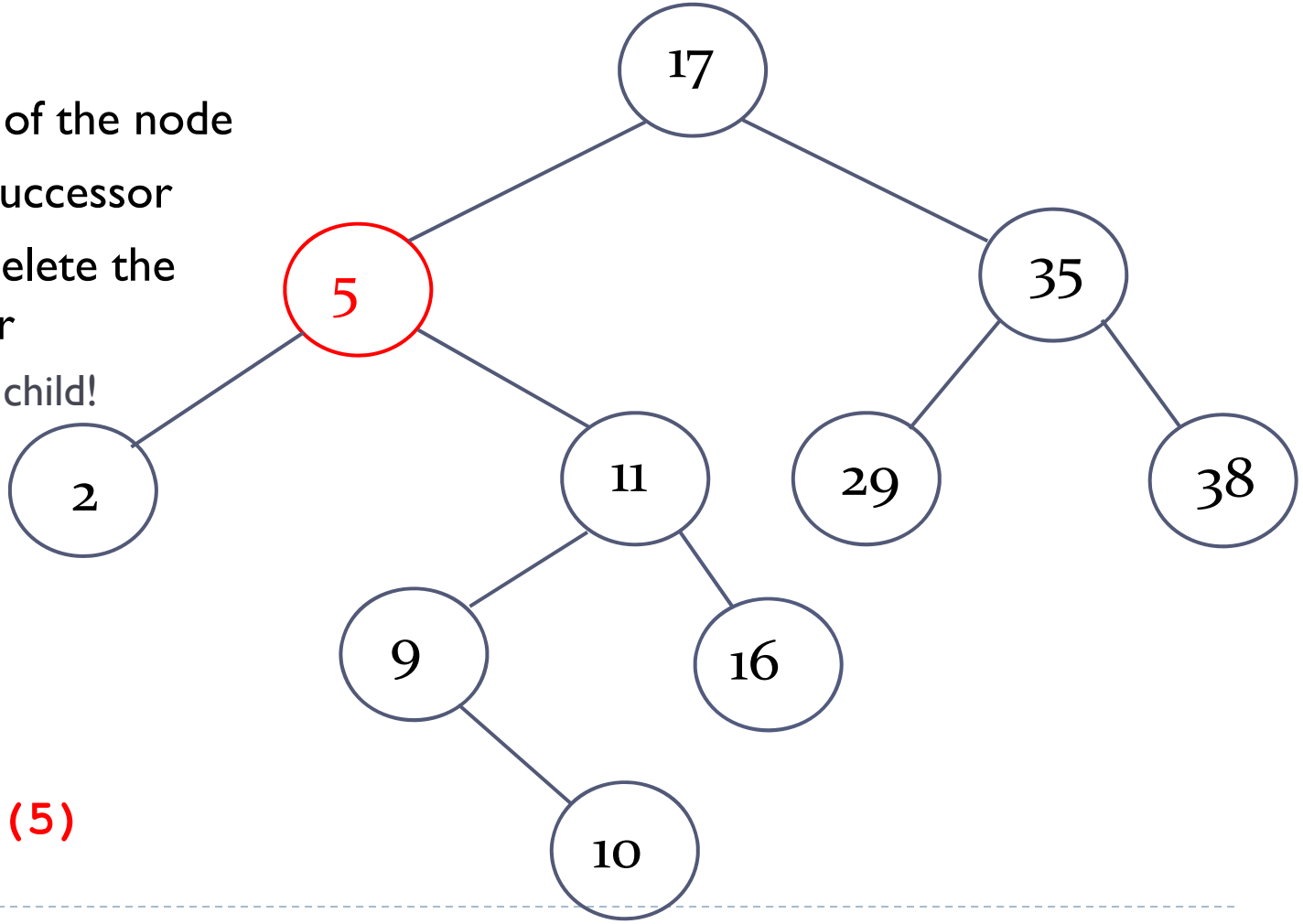
The in-order successor of 14?

The in-order successor of 70?

Binary search trees - deleting

Case 3: Deleting a node with two children

- Find the node
- Replace the value of the node with its in-order successor
- We also have to delete the in-order successor
  - Has at most one child! Think why!

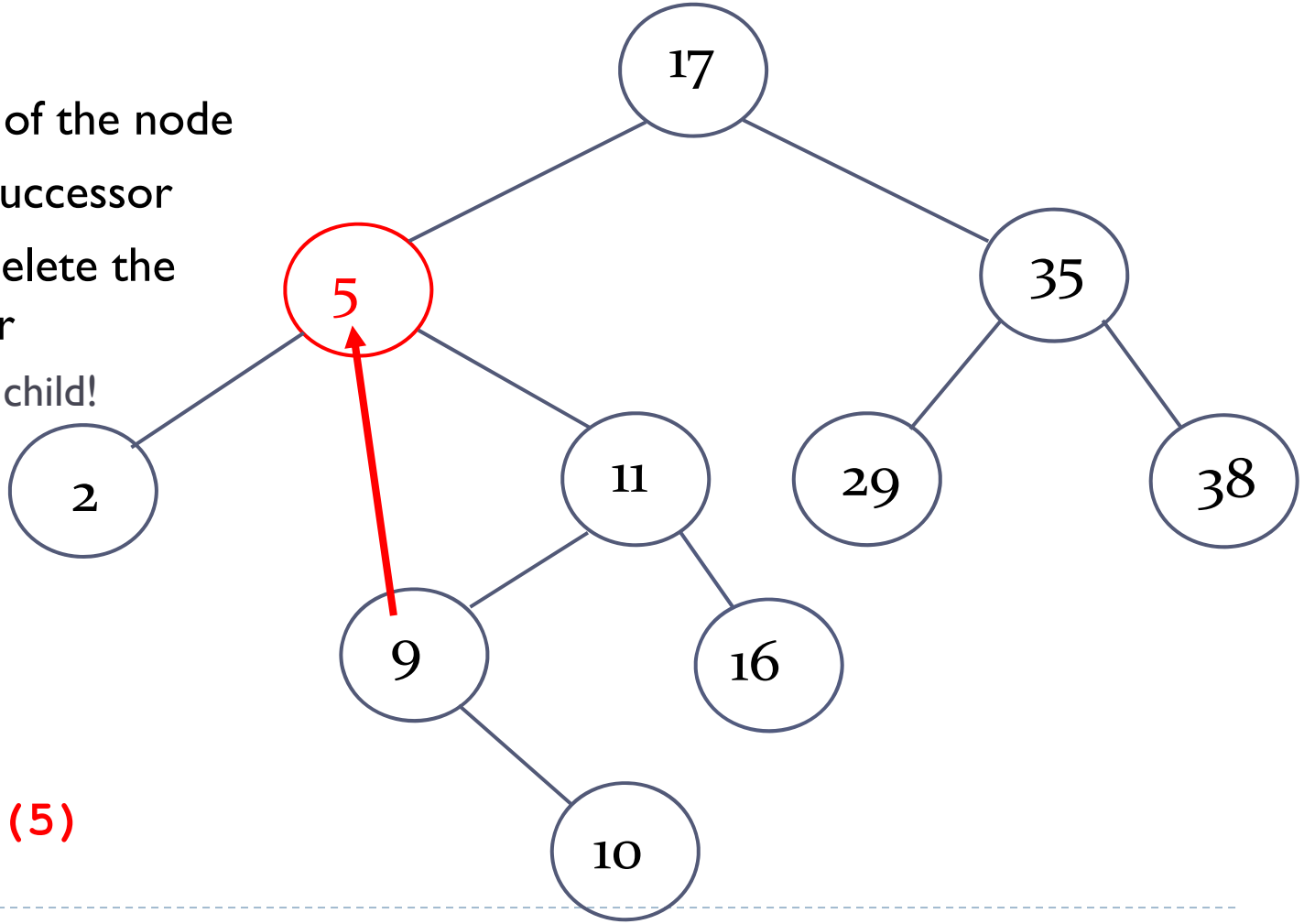



```
a_tree.delete(5)
```

Binary search trees - deleting

Case 3: Deleting a node with two children

- Find the node
- Replace the value of the node with its in-order successor
- We also have to delete the in-order successor
  - Has at most one child!

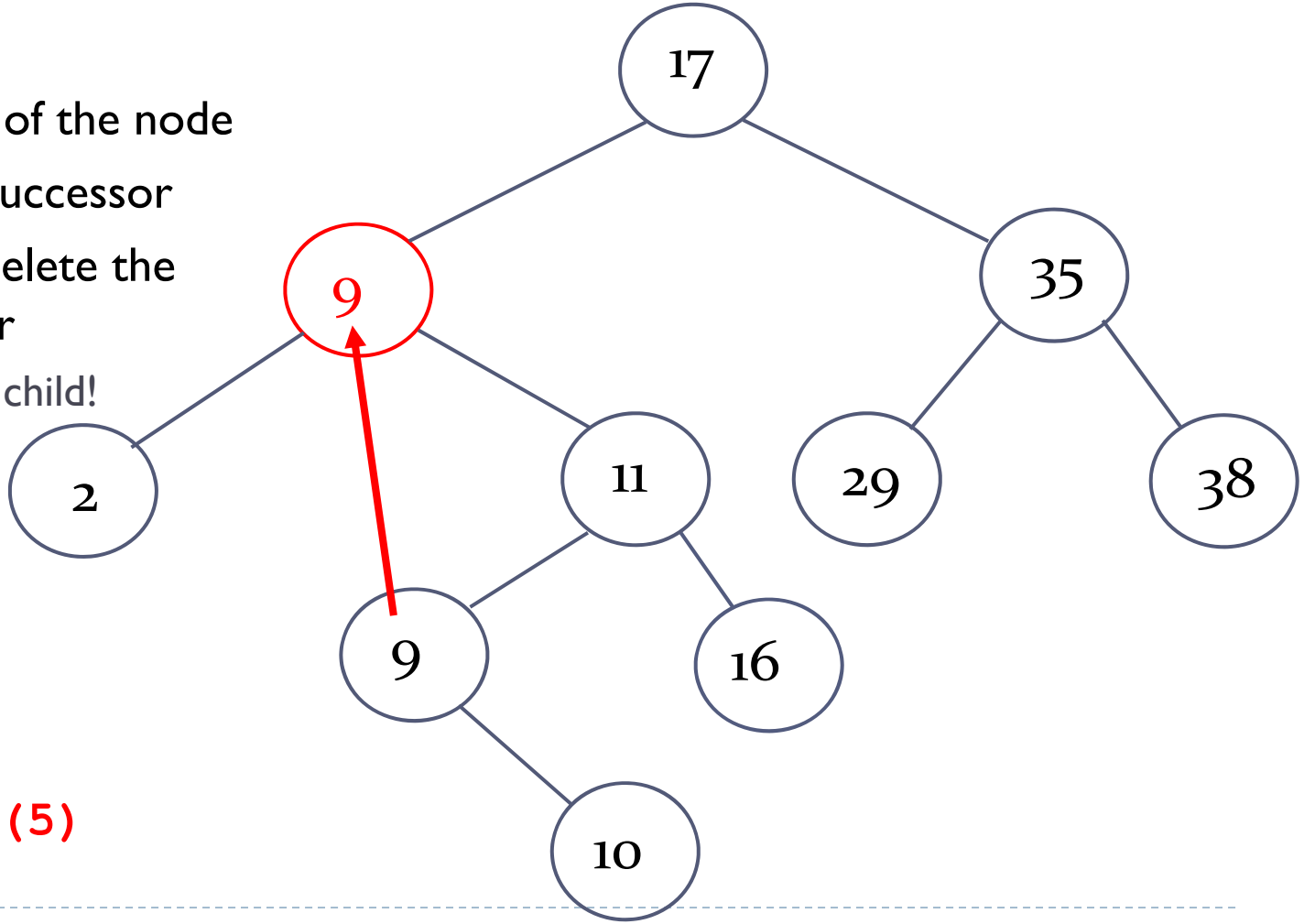



```
a_tree.delete(5)
```

Binary search trees - deleting

Case 3: Deleting a node with two children

- Find the node
- Replace the value of the node with its in-order successor
- We also have to delete the in-order successor
  - Has at most one child!

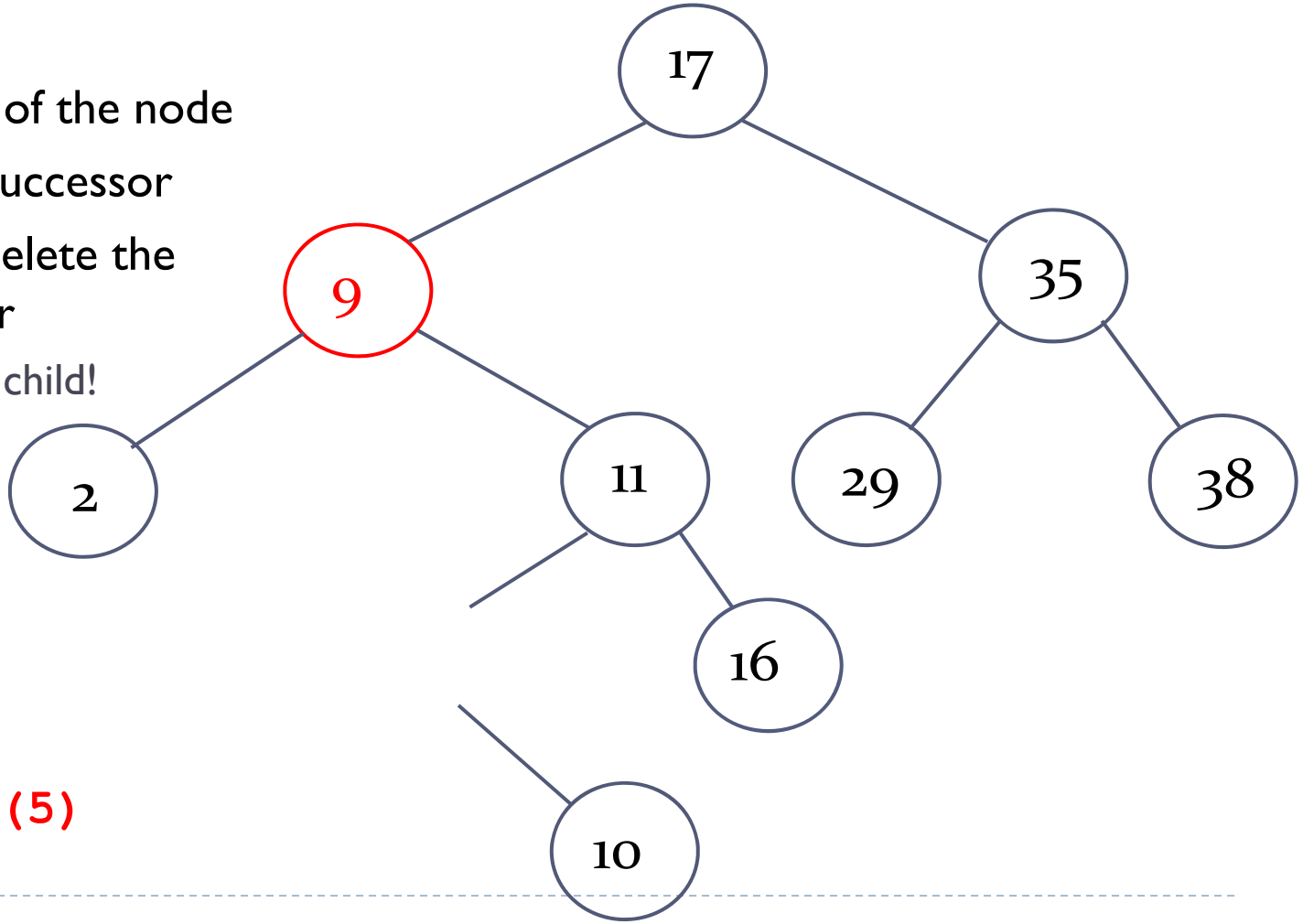



```
a_tree.delete(5)
```

Binary search trees - deleting

Case 3: Deleting a node with two children

- Find the node
- Replace the value of the node with its in-order successor
- We also have to delete the in-order successor
  - Has at most one child!

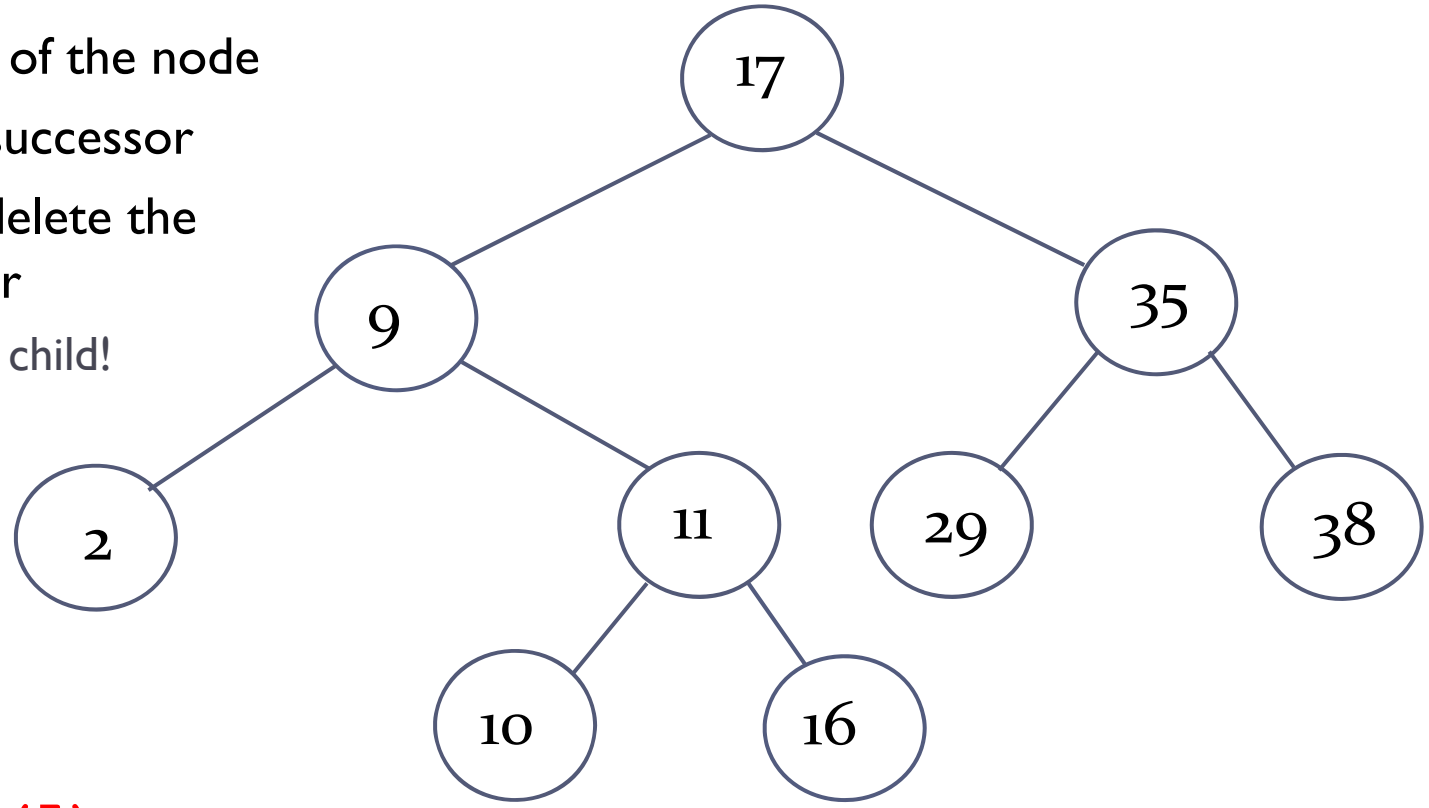



```
a_tree.delete(5)
```

Binary search trees - deleting

Case 3: Deleting a node with two children

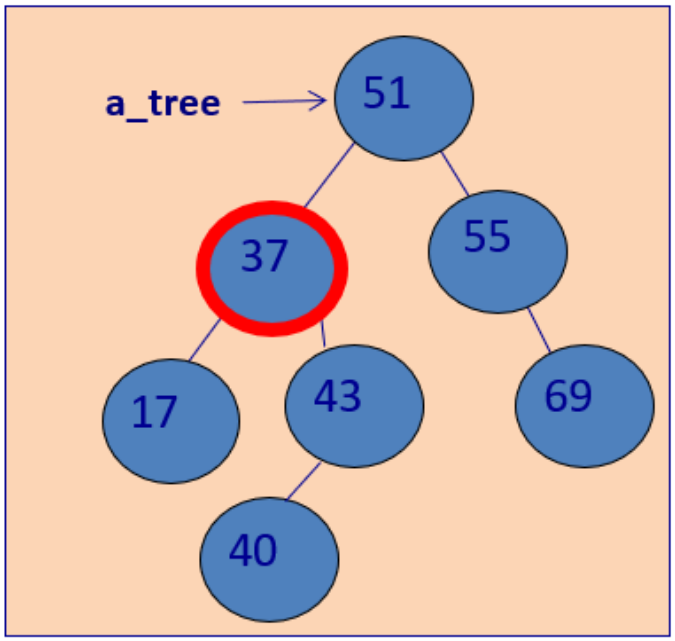
- Find the node
- Replace the value of the node with its in-order successor
- We also have to delete the in-order successor
  - Has at most one child!




```
a_tree.delete(5)
```

Binary search trees - deleting

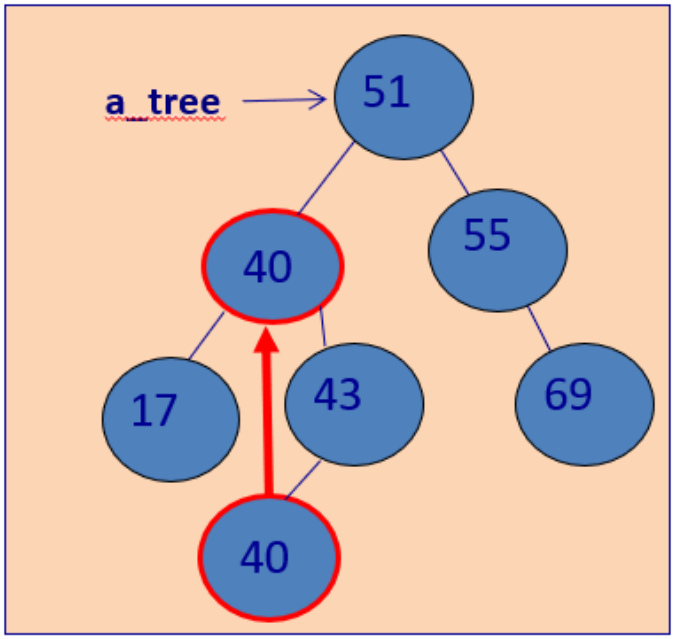
- **Case 3:** Deleting a node with two children
- Another example




```
a_tree.delete(37)
```

Binary search trees - deleting

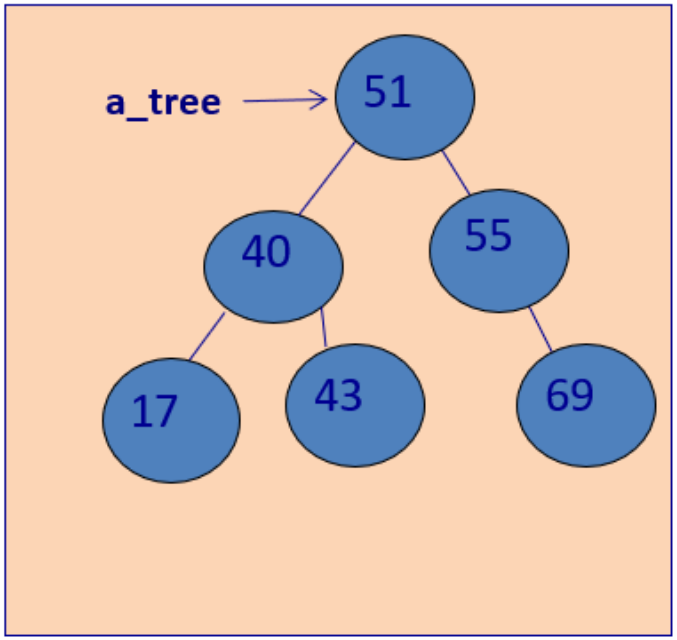
- **Case 3:** Deleting a node with two children
- Another example




```
a_tree.delete(37)
```

Binary search trees - deleting

- **Case 3:** Deleting a node with two children
- Another example




```
a_tree.delete(37)
```

Parent pointer

For some operations, such as deletion, it is convenient to have a parent pointer.

```
class BinarySearchTree:
    def __init__(self, data, left=None, right=None, parent = None):
        self.data = data
        self.left = left
        self.right = right
        self.parent = parent
```

Performance of BST - Complexity

```
NOTE: A tree is balanced if for every node its left
and right subtree vary in height by at most one
```

If a Binary Search Tree is balanced then height is O(log n) and hence insert, search, delete are all O(log n)!

One can show that average running times for insert, search, delete are all O(log n)!

Worst case is O(n)

BUT: Can create tree which is always balanced and hence always O(log n)

Advantages of BST

Compared to **unsorted list**:

- Insert is slower (O(log n) vs. O(1)), but delete and find are much faster (O(log n) vs. O(n))

Compared to **sorted list**:

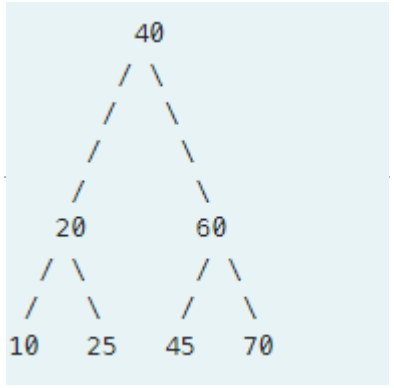
- Both have O(log n) find operation, but BST can also insert and delete in O(log n)

Compared to **hashtable**:

- Don't need to find a suitable hash function
- Low memory overhead
- No need for expensive operations when increasing size (=> rehashing)
- Maintains an order => can list elements in sorted order
  => can find certain subsets efficiently

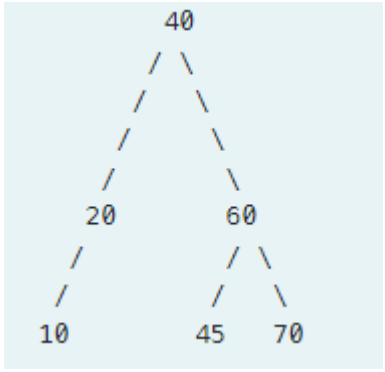
Exercise 3 - 4

- Q3 Given the following BST:



  - What does the tree look like after removing a node with the value 25?

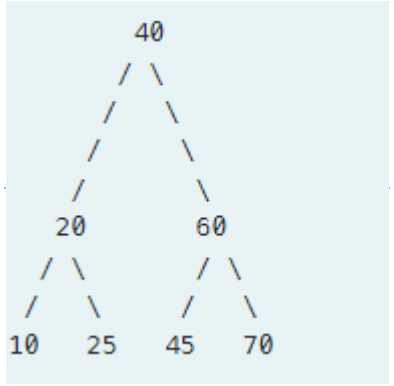
- Q4 Given the following BST:



  - What does the tree look like after removing a node with the value 20?

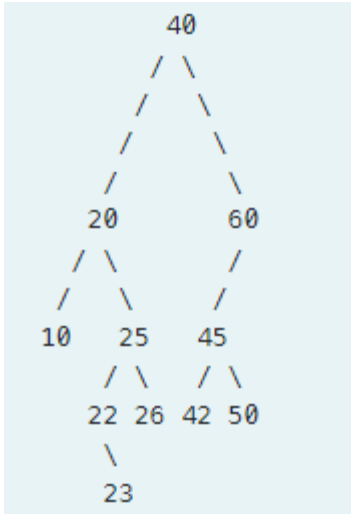
Exercise 5 - 6

- Q5 Given the following BST:



  - What does the tree look like after removing a node with the value 60?

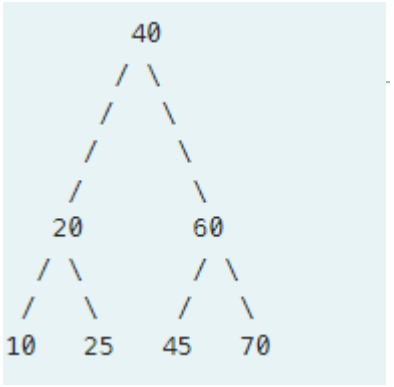
- Q6 Given the following BST:



  - What does the tree look like after removing a node with the value 20?

Exercise 7

- Q7: Given the following BST:



  - What does the tree look like after removing the root?

Exercise 8 – tree traversals

The nodes of the tree can be traversed in different orders.

in-order visits the tree:

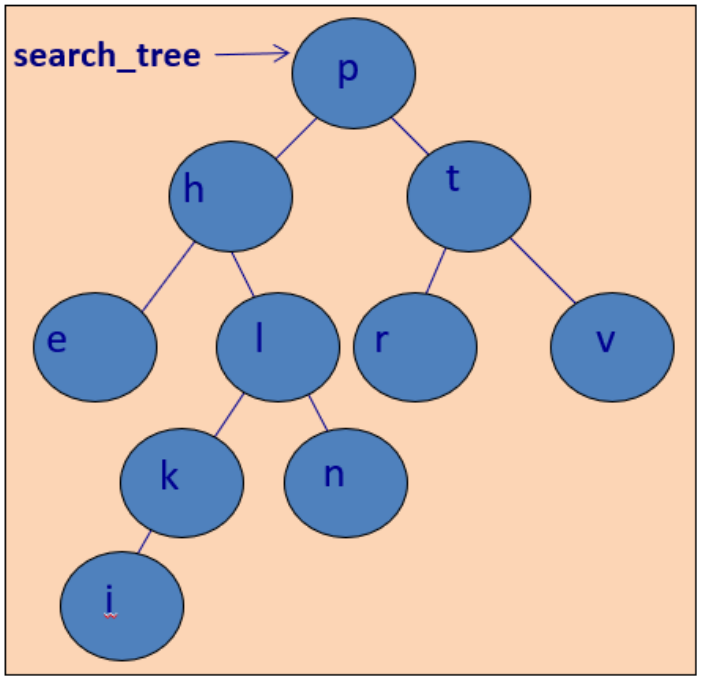
left
node
right

pre-order visits the tree:

node
left
right

post-order visits the tree:

left
right
node



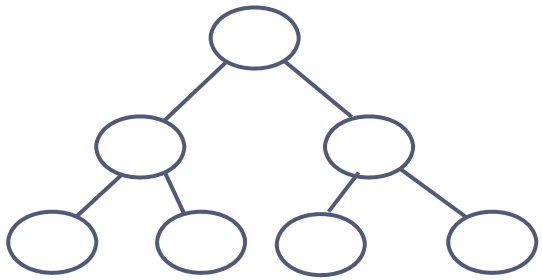
Level order visits the tree:
Left to right, level by level

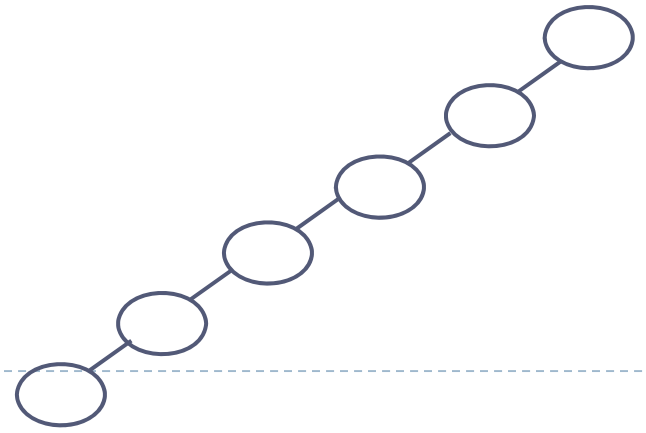
Extra – Tree Sort

Define an algorithm for sorting n values using a BST
What is its running time?

Algorithm:
(1) Insert n values into initially empty Binary Search Tree
(2) Output values by performing an in-order traversal

Running time:
Best case: resulting tree is (almost) balanced => O(nlog n)
Worst case: resulting tree is linear (linked list) => O(n^2)

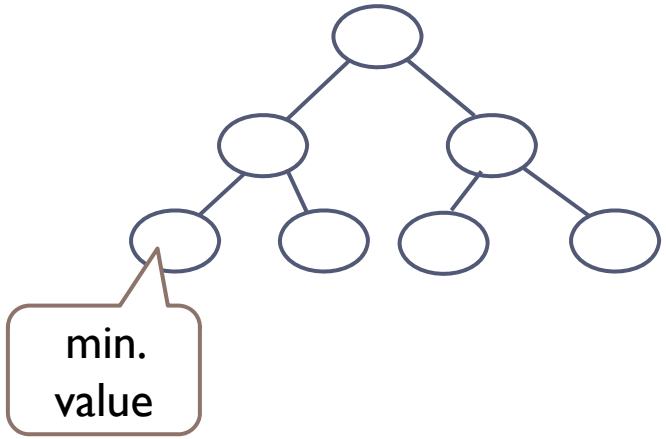




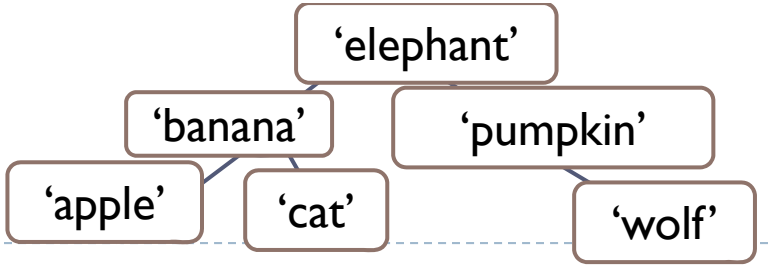
Exercise 9

- Define a function named get_min(bst) which takes a binary search tree as a parameter. The function returns the smallest value in the parameter binary search tree

```
def get_min(bst)
```



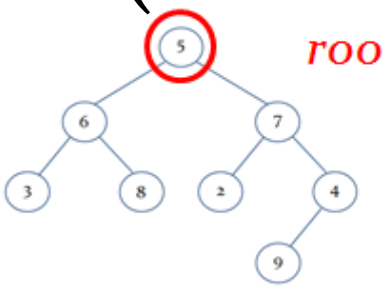
Exercise 10



- Define a function named get_shortest_word(bst)
  - returns the shortest word in the binary search tree.
  - returns None if the tree is empty

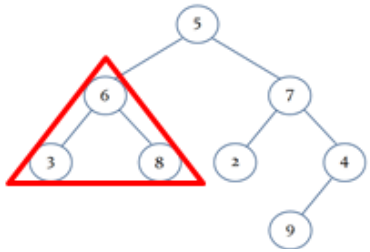
```
def get_shortest_word(bst):
    if
```

min(



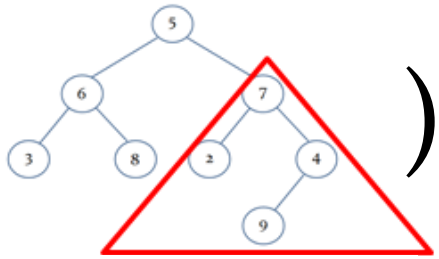
root node

,



shortest word
of left sub-tree

,



shortest word
of right sub-tree

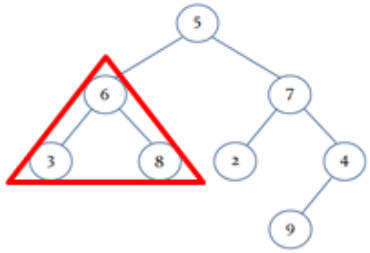
)

Exercise 11

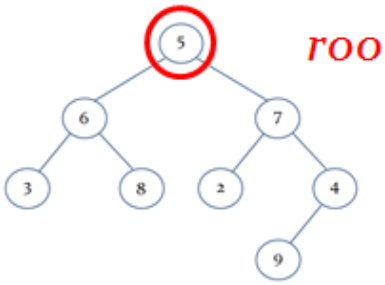
- Define a function called get_bst_inorder(tree)
  - returns a Python list containing values in the in-order traversal of the parameter binary search tree.

```
def get_bst_inorder(bst):
    if
```

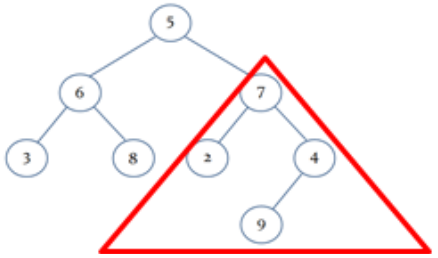
`print(get_bst_inorder(tree1))`	`[1, 2, 5, 7, 9]`



left subtree



root node



right subtree